

Say we have the following snippet of code to use as a basis.

```
void Test(int n)
{
    if (n > 0)
    {
        printf(n);
        Test(n-1);
    }
}
```

And now before we begin with the actual recurrence relations, let's trace it to see how this works. Say $n = 3$.

$3 > 0$? yes → print 3. then Test(2) → $2 > 0$? yes → print 2. Test(1) → $1 > 0$? yes → print 1. Test(0) → $0 > 0$? No. STOP!!

```
void Test(int n) → T(n)
{
    if (n > 0) → can ignore this or just say 1. doesn't matter
    {
        printf(n); → 1
        Test(n-1); → T(n-1)
    }
}
```

$$T(n) = \begin{cases} 1 & , \quad n = 0 \\ T(n-1) + 1 & , \quad n > 0 \end{cases}$$

$$T(n) = T(n-1) + 1$$

$$T(n-1) = T(n-2) + 1$$

So now let's substitute $T(n-1)$

$$T(n) = T(n-1) + 1$$

$$T(n) = [T(n-2) + 1] + 1$$

$$T(n) = T(n-2) + 2$$

∴ we can continue this process and we ultimately see that it's just $T(n) = T(n-k) + k$

now let's assume that

$$n - k = 0$$

$$n = k$$

we do this b/c when $n = 0$, $T(n) = 1$, it's the smallest value we can substitute

$$T(n) = T(n-k) + k$$

$$T(n) = T(n-n) + n$$

$$T(n) = T(0) + n$$

$$= 1 + n$$

$$\boxed{O(n)}$$



Now we have the following code

```
void Test(int n) —————  $T(n)$ 
{
  if (n > 0) —————> ignore or 1
  {
    for (i = 0; i < n; i++) —————>  $n+1$ 
    {
      printf(n) —————>  $n$ 
    }
    Test(n-1) —————>  $T(n-1)$ 
  }
}
```

So in total, we have

$$T(n) = T(n-1) + 2n + 2$$

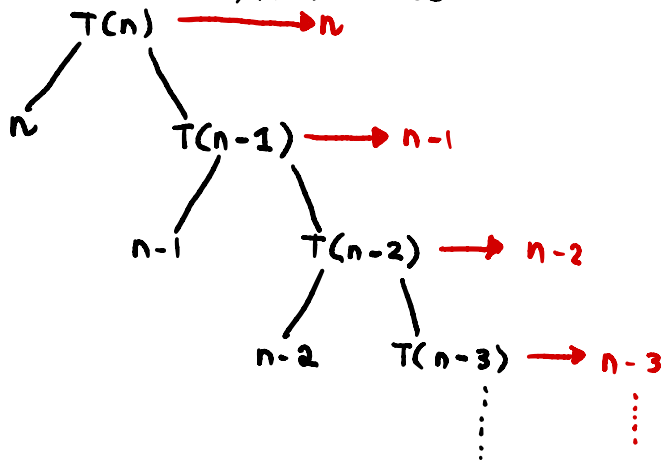
this itself is hard to write a recurrence relation + annoying to substitute. So we can rewrite this as just n .

$$T(n) = T(n-1) + n$$

thus, just like before, we have

$$T(n) = \begin{cases} 1, & n = 0 \\ T(n-1) + n, & n > 0 \end{cases}$$

To better visualize it, make a tree



So if we add all of these up, we'll then have

$$n + n-1 + n-2 + n-3 + \dots, \text{ which results in } \frac{n(n+1)}{2} \rightarrow \frac{n^2 + n}{2} \rightarrow \boxed{O(n^2)}$$

Say we now have the following code

```
void Test(int n) —————  $T(n)$ 
{
  if (n > 0) ————— 1
  {
    for (i = 1; i < n; i = i * 2) —  $\log_2 n$ 
    {
      print(i) —————  $\log(n)$ 
    }
    Test(n-1) —————  $T(n-1)$ 
  }
}
```

$$T(n) = T(n-1) + \log n$$

we don't need these two b/c it'll ultimately lead to the same answer, just with more steps. So what Abdul Bari does is cancel them.

So now the recurrence relation that we have to make is

$$T(n) = \begin{cases} 1 & , n = 0 \\ T(n-1) + \log n & , n > 0 \end{cases}$$

Final answer is $O(n \log n)$

CONCLUSION ★★ ★ IMPORTANT TO KNOW!

↳ based on the three videos in this miniseries, there are some important patterns and tips to keep in mind

$$\begin{aligned} T(n) &= T(n-1) + 1 \rightarrow O(n) \\ T(n) &= T(n-1) + n \rightarrow O(n^2) \\ T(n) &= T(n-1) + \log n \rightarrow O(n \log n) \end{aligned}$$

Notice how whatever's after the $(n-1)$, we can just multiply it by n and that's our answer?
 $1 \cdot n = n$, $n \cdot n = n^2$, $\log n \cdot n = n \log n$.

regardless of how many steps there are, we still refer to this pattern

$$\begin{aligned} T(n) &= T(n-2) + 1 \rightarrow \frac{n}{2} = O(n) \\ T(n) &= T(n-1000) + n \rightarrow O(n^2) \end{aligned}$$

Say we have the following code of

Algorithm Test (int n) $\rightarrow T(n)$

```

{ if (n > 0)
{ printf(n);  $\rightarrow 1$ 
  Test(n-1);  $\rightarrow T(n-1)$ 
  Test(n-1);  $\rightarrow T(n-1)$ 
}
}
    
```

$$T(n) = 2T(n-1) + 1$$

now the recurrence relation will fully be

$$T(n) = \begin{cases} 1, & n=0 \\ 2T(n-1) + 1, & n > 0 \end{cases}$$

$$T(n) = 2T(n-1) + 1$$

$$T(n) = 2[2T(n-2) + 1] + 1$$

we get this by
knowing what $T(n-1)$
was in the first video

$$T(n) = 2^2 T(n-2) + 2 + 1$$

now solve
for this

we ultimately
get $O(2^n)$